How the Interval Call Forecast Was Built

A plain-language, step-by-step walkthrough of the logic — no coding background needed

Prepared by Momna Ali | For: Aradhna | Pseudocode walkthrough — not real code, just the logic in plain steps

How to Read This Document

This walks through the actual logic behind the interval call forecast, in plain steps — not real Python code, just what each piece of code is doing and why. Anywhere you see a shaded box, that's the "almost-English" version of a real script: read it top to bottom like a recipe, not like code you need to type.

Every section has a "Why this step" note — that's usually the more important part. The logic itself is often simple; the reasoning behind why we did it a particular way is where the real thinking happened.

Part 1: Turning Raw Calls Into 15-Minute Buckets

We started with a table of individual phone calls — one row per call, with an exact timestamp and which queue it went to. The goal was to turn that into "how many calls happened in each 15-minute window," for each queue, for each day.

```
for every call in the raw call table:
    round its timestamp DOWN to the nearest 15-minute mark
        (example: a call at 9:07am and a call at 9:14am
         both belong to the "9:00am bucket")

group all calls by: which queue, which day, which 15-min bucket
count how many calls landed in each group

result: one row per (queue, day, 15-min bucket) with a call count
```

> ***Why this step:*** *this is the foundation everything else is built on. If the buckets are wrong, every number downstream is wrong too — so this step was checked carefully against real sample data before moving on.*

Part 2: Filling In the Missing Zeros

Here's a subtle trap: the raw data only has rows for buckets where at least one call happened. If a queue had zero calls between 2:00am and 2:15am, there's simply no row for it — it's not there at all, not marked as zero.

If we just averaged the rows we had, we'd be secretly ignoring every quiet 15-minute window and only averaging the busy ones — which would make every forecast number too high.

```
for each queue:
    figure out every 15-minute time-of-day this queue has
    EVER shown up at (its real "operating hours")

    build a full grid: every real day  x  every real time-of-day
    (this grid has a row for every possible combination,
     even ones that never had a single call)

    match the real call counts onto this grid
```

```
    anywhere that didn't match → fill in with 0

result: a complete dataset with TRUE zeros included,
not just the moments that happened to have calls
```

> ***Why this step:*** *without this step, every average would be biased upward. This is the same kind of "quiet bug" that caused real problems in an earlier project (the daily call forecast), so we built the fix in from the start this time instead of finding it later.*

Part 3: Building the Baseline Forecast

Once we had the true zero-filled data, building the actual forecast was straightforward: average call volume by day-of-week and time-of-day.

```
for each queue:
    group all rows by (day of week, time-of-day bucket)
        example group: "all the Tuesdays at 9:00am"

    for each group:
        ForecastVolume = average call count in that group
        StdDev = how much that group's calls typically vary
        SampleSize = how many weeks of data went into it

result: for every queue, every day of week, every 15-min slot —
a forecasted number, plus how much it normally wobbles,
plus how many weeks of history that number is based on
```

> ***Why this step:*** *we kept the StdDev and SampleSize numbers on purpose, not just the forecast itself. A forecast without knowing how reliable it is can be misleading — these two extra numbers are what let us later build an honest deviation alert, and flag which forecasts have less history behind them.*

Part 4: Testing the Forecast Honestly (Leave-One-Out)

Before trusting any accuracy number, we had to test the forecast fairly. The wrong way to test: check the forecast against the same days it was built from — that always makes it look better than it really is, because the answer is baked into the average.

The right way — leave-one-out — tests each day using a forecast built from every day EXCEPT that one.

```
for each real day in the data:
    temporarily remove that day from its group
    (example: remove THIS Tuesday from all the other Tuesdays)

    recalculate the average using only the remaining days
    → this becomes the "honest forecast" for the removed day

    compare: honest forecast vs. what actually happened that day
    record the size of the error

after doing this for every single day:
    MAPE = the average error size, as a percent
    Within15pct = what % of days were close enough (within 15%)
```

Why this step: *this is the single most important habit in this whole project. Any time you hear a number like '47.9% average error,' it should have come from a leave-one-out test like this — otherwise the number can't really be trusted.*

Part 5: The Hybrid Method (and Why It Didn't Help)

We also tried a second, more advanced approach: instead of forecasting each 15-minute slot directly, forecast the whole day's total first, then split it using each time slot's usual share of the day.

```
for each queue:
    STEP A — forecast the DAILY total
        (same leave-one-out idea, but for the whole day's calls
         instead of one 15-min slot)

    STEP B — forecast each time slot's SHARE of the day
        example: "Tuesdays at 9am usually make up 4% of the day's calls"
        (also computed leave-one-out, so it's not cheating either)

    STEP C — combine them:
        HybridForecast = DailyForecast  x  ShareForecast

    test THIS forecast the same honest way as Part 4
```

Why this step: *this turned out to be a genuinely useful negative result. It scored almost identically to the simple method (48.0% vs. 47.9%), which told us the real problem isn't how calls are distributed across the day — it's that a single queue's daily total is noisy all on its own. Knowing what DOESN'T work narrows down what might.*

Part 6: Comparing Different Time-Bucket Sizes

Since 15-minute buckets were noisy, we tested whether wider buckets (30 minutes, a full hour) would be more accurate — built directly from the same 15-minute data, just added together in bigger groups.

```
for bucket_size in [15 minutes, 30 minutes, 60 minutes]:

    re-group the original 15-min data into the new bucket size
    (example: for 30-min buckets, add together each pair
     of neighboring 15-min buckets)

    re-do Parts 2, 3, and 4 on this newly-grouped data
    (fill zeros → build baseline → test honestly)

    record the accuracy (MAPE) for this bucket size

compare all three accuracy numbers side by side
```

Why this step: *this answers a real business tradeoff, not just a technical curiosity: bigger buckets are more accurate, but tell staffing less precisely WHEN to expect the calls. 30-minute buckets turned out to be the accuracy sweet spot — better than both 15-minute and hourly.*

Part 7: The Deviation Alert (Catching Real Spikes)

The final piece: a way to flag when a real day's actual call volume looks unusual compared to what the forecast expected — using a z-score (how many “typical wobbles” away from normal something is) instead of one flat rule for every queue.

```
for every real 15-minute window we have:

    look up the ForecastVolume and StdDev for that exact
    (queue, day of week, time slot) from Part 3

    Deviation = Actual − ForecastVolume
    ZScore = Deviation / StdDev
        (this turns a raw difference into “how unusual is this,
         relative to how much this specific slot normally wobbles”)

    IF the forecast itself is too small to trust (under 3 calls):
        SKIP this window — too sparse to alert on reliably
    ELSE IF |ZScore| is 2.5 or more:
        FLAG this window as an anomaly
```

> ***Why this step:*** *using a z-score instead of a flat percentage was a deliberate choice. Some time slots naturally swing a lot (like a busy 10am peak) and some barely move at all (like a quiet 7am slot) — a single flat rule (like “flag anything 30% off”) would constantly false-alarm on the busy slots and miss real problems on the quiet ones. The z-score automatically adjusts to each slot's own normal behavior. We also added the “skip if too sparse” rule after finding a real bug: for a queue that's almost always at zero calls, even a single call arriving would otherwise look like an extreme, alarming spike — when it's actually nothing unusual at all.*

Part 8: Handling the Low-Volume Queues Differently

Not every queue had enough calls to support a 15-minute forecast. For those, we used a much simpler approach: just the average calls PER DAY, broken down by day of week — no time-of-day detail at all.

```
for each low-volume queue:
    group all calls by DAY only (not by 15-min slot)
    fill in zero-call days the same honest way as Part 2
    average by day of week
    label the result clearly as "low volume — directional only"
```

> ***Why this step:*** *forcing a detailed 15-minute forecast onto a queue that gets 2-3 calls a day would have produced a number that looks precise but means almost nothing — most windows would show zero, with no real pattern underneath. Being honest about a queue's limits is more useful than pretending a model can do more than the data actually supports.*

A Quick Glossary

- MAPE (Mean Absolute Percentage Error): the average size of the forecast's error, as a percentage. Lower is better.
- Leave-one-out: a way of testing a forecast fairly, by never letting it “peek” at the exact day it's being tested against.

- Standard deviation (StdDev): a measure of how much a number normally bounces around. A small StdDev means a value is usually stable; a large one means it swings a lot.
- Z-score: how many “standard deviations” away from normal something is. A z-score of 0 is perfectly average; a z-score of 3 is quite unusual.
- Baseline: the plain, first-pass forecast (day-of-week + time-of-day average) that everything else in this project was tested against.